Topico 4: Capacidade de Canal e Codigos Corretores de Erro

Cenario: link de radio Terra-Marte (canal binario simetrico). Enunciado: `propostas/1_Simulacoes.pdf`, secao 4.

Preparacao

In [1]:
```
import sys, pathlib

REPO = pathlib.Path.cwd().parents[2]          # .../<topico>/ -> .../<proposta>/ -> raiz
if str(REPO) not in sys.path:
    sys.path.insert(0, str(REPO))

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from PIL import Image
from scipy.stats import binom

from utils.info_theory import (
    entropy, binary_entropy, joint_entropy, conditional_entropy,
    cross_entropy, kl_divergence, mutual_information,
    conditional_mutual_information, mi_gtest_statistic,
)
from utils.simulation import (
    rng, sample_pmf, sample_joint, empirical_pmf, bsc, markov_chain,
)
from utils.estimation import bootstrap_ci
from utils.plotting import setup_estilo, salvar_fig, plot_curvas, PALETA, salvar_tabela
import src.model as model                     # helpers especificos deste topico

setup_estilo()
GEN = rng(20260902)                           # seed fixo -> resultados reproduziveis
DADOS = pathlib.Path("dados")
FIGURAS = pathlib.Path("figuras")
ENTRADA = pathlib.Path("entrada")             # ativos de entrada versionados (ex.: sample.jpg)
```

Passo 1: Canal BSC

Modelar o canal com parametro p (algo entre 0.05 e 0.2).

In [2]:
```
# Passo 1
P_CANAL = 0.1  # probabilidade de erro por uso do canal (link Terra-Marte ruidoso)

print(f"p do canal binario simetrico = {P_CANAL}")
```

```
p do canal binario simetrico = 0.1
```

Passo 2: Capacidade teorica

C = 1 - binary_entropy(p), em bits por uso do canal.

In [3]:
```
# Passo 2
H_B_P = binary_entropy(P_CANAL)
C = 1 - H_B_P

print(f"H_b(p) = {H_B_P:.4f} bits")
print(f"capacidade C = 1 - H_b(p) = {C:.4f} bits por uso do canal")
```

```
H_b(p) = 0.4690 bits
capacidade C = 1 - H_b(p) = 0.5310 bits por uso do canal
```

Passo 3: Mensagem

Se houver uma imagem em `entrada/sample.jpg` (versionada, diferente de `dados/` / `figuras/`), carrega-la, aplicar um filtro de binarizacao (limiar sobre a escala de cinza) e usar os pixels binarios como sequencia de bits de informacao -- imagem de verdade em vez da mensagem sintetica, bem mais bits, util para checar a performance dos passos seguintes em escala maior. Sem essa imagem (caso raro, ex.: `entrada/` removida localmente), o passo cai de volta para a mensagem sintetica original.

In [4]:
```
# Passo 3
CAMINHO_IMAGEM = ENTRADA / "sample.jpg"


def binariza_por_limiar(imagem_cinza: np.ndarray, limiar: float) -> np.ndarray:
    """Filtro de binarizacao: cada pixel vira 1 se estiver acima do limiar."""
    return (imagem_cinza > limiar).astype(np.uint8)


if CAMINHO_IMAGEM.exists():
    IMAGEM_CINZA = np.asarray(Image.open(CAMINHO_IMAGEM).convert("L"))
    LIMIAR = IMAGEM_CINZA.mean()  # limiar simples: media de cinza da propria
imagem
    IMAGEM_BINARIA = binariza_por_limiar(IMAGEM_CINZA, LIMIAR)

    fig, eixos = plt.subplots(1, 2, figsize=(10, 5))
    eixos[0].imshow(IMAGEM_CINZA, cmap="gray")
    eixos[0].set_title("original (escala de cinza)")
    eixos[0].axis("off")
    eixos[1].imshow(IMAGEM_BINARIA, cmap="gray")
    eixos[1].set_title(f"binarizada (limiar = {LIMIAR:.1f})")
```

```
    eixos[1].axis("off")
    salvar_fig("t4_passo3_imagem_binarizada")
    plt.show()

    MSG_COMPLETA = IMAGEM_BINARIA.ravel()
    N_BITS = MSG_COMPLETA.size - (MSG_COMPLETA.size % 4)  # Hamming(7,4) exige
multiplo de 4
    MSG = MSG_COMPLETA[:N_BITS]
    SOBRA_IMAGEM = MSG_COMPLETA[N_BITS:]  # poucos bits cortados; guardados para
reconstruir a imagem no Passo 10
    print(f"mensagem: imagem real
({IMAGEM_CINZA.shape[1]}x{IMAGEM_CINZA.shape[0]})")
else:
    # sem imagem local (caso raro) -> mensagem sintetica de sempre
    N_BITS = 200_000  # pixels de uma imagem simplificada, preto/branco
equiprovaveis
    MSG = sample_pmf([0.5, 0.5], size=N_BITS, generator=GEN).astype(np.uint8)
    print("mensagem: sintetica (entrada/sample.jpg nao encontrada)")

print(f"N_BITS = {N_BITS:,}")
print(f"primeiros 20 bits: {MSG[:20]}")
print(f"fracao de 1s na mensagem: {MSG.mean():.4f}")
```




```
mensagem: imagem real (2570x3427)
N_BITS = 8,807,388
primeiros 20 bits: [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
fracao de 1s na mensagem: 0.5204
```

Passo 4: Transmissao direta

Passar os bits pelo `bsc` sem codificacao; medir a BER e comparar com p.

In [5]:
```
# Passo 4
RECEBIDO_DIRETO = bsc(MSG, P_CANAL, generator=GEN)
```

```
ber_direto = np.mean(RECEBIDO_DIRETO != MSG)

print(f"BER sem codificacao: {ber_direto:.4f}")
print(f"p do canal:          {P_CANAL:.4f}")
print(f"diferenca:           {abs(ber_direto - P_CANAL):.4f}")
```

```
BER sem codificacao: 0.1001
p do canal:          0.1000
diferenca:           0.0001
```

Passo 5: Codigo de repeticao

`model.repetition_encode` / `repetition_decode` (n_rep impar); medir a taxa efetiva R = 1/n_rep e a BER pos-decodificacao.

In [6]:
```
# Passo 5
N_REP = 3  # impar -> voto de maioria sem empate

COD_REP = model.repetition_encode(MSG, N_REP)
RECEBIDO_REP = bsc(COD_REP, P_CANAL, generator=GEN)
DECOD_REP = model.repetition_decode(RECEBIDO_REP, N_REP)

R_REP = 1 / N_REP
ber_rep = np.mean(DECOD_REP != MSG)

print(f"n_rep = {N_REP}")
print(f"taxa efetiva R = 1/n_rep = {R_REP:.4f}")
print(f"BER pos-decodificacao (repeticao): {ber_rep:.6f}")
```

```
n_rep = 3
taxa efetiva R = 1/n_rep = 0.3333
BER pos-decodificacao (repeticao): 0.028023
```

Passo 6: Hamming(7,4)

`model.hamming74_encode` / `hamming74_decode` (decodificacao por sindrome); medir R = 4/7 e a BER.

In [7]:
```
# Passo 6
COD_HAM = model.hamming74_encode(MSG)
RECEBIDO_HAM = bsc(COD_HAM, P_CANAL, generator=GEN)
DECOD_HAM = model.hamming74_decode(RECEBIDO_HAM)

R_HAM = 4 / 7
ber_ham = np.mean(DECOD_HAM != MSG)

print(f"taxa efetiva R = 4/7 = {R_HAM:.4f}")
print(f"BER pos-decodificacao (Hamming(7,4)): {ber_ham:.6f}")
```

```
taxa efetiva R = 4/7 = 0.5714
BER pos-decodificacao (Hamming(7,4)): 0.066832
```

Passo 7: Tabela comparativa

BER e taxa efetiva das tres abordagens para o mesmo p, e para cada uma se R > C e o piso teorico de BER que a parte inversa do teorema impoe a qualquer codigo daquela taxa.

In [8]:
```
# Passo 7
TABELA_ESQUEMAS = pd.DataFrame(
    {
        "esquema": ["sem codificacao", f"repeticao (n={N_REP})", "Hamming(7,4)"],
        "R": [1.0, R_REP, R_HAM],
        "BER": [ber_direto, ber_rep, ber_ham],
    }
).set_index("esquema")

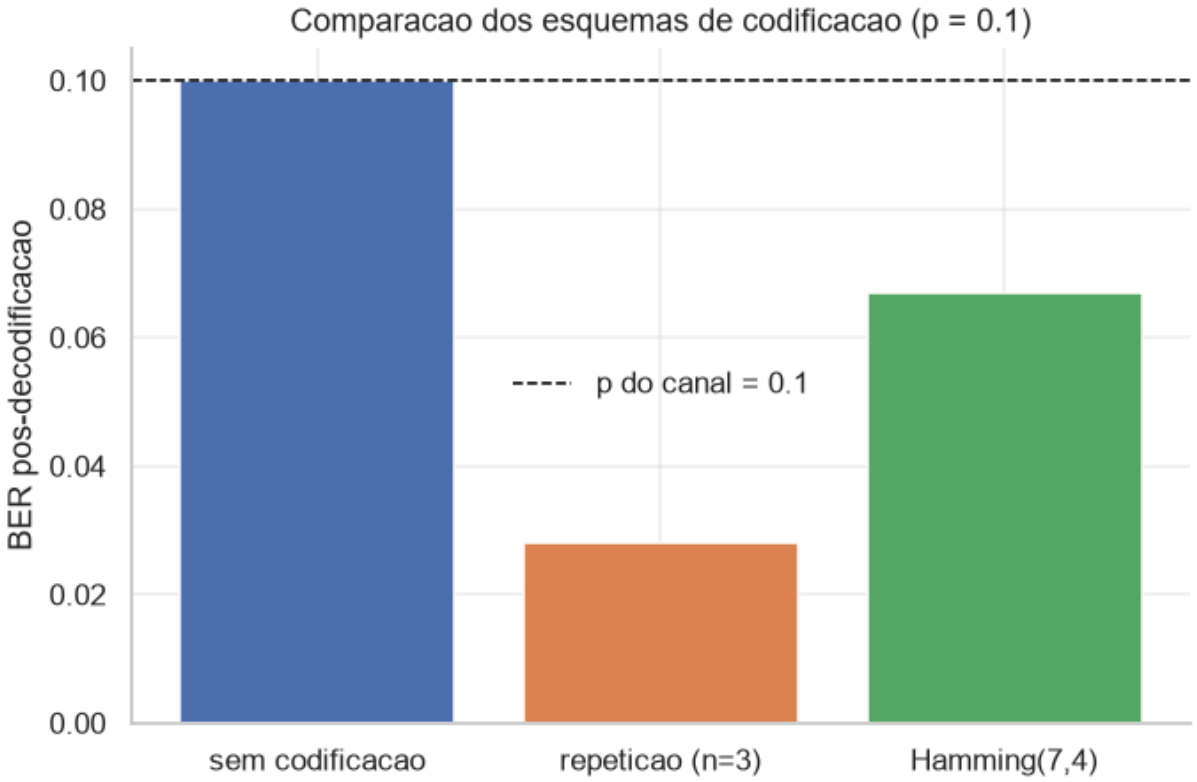
# R > C nao e so o cenario do Passo 10: com p = 0.10, C = 0.531, e o
# Hamming(7,4) (R = 4/7 = 0.571) ja opera ACIMA da capacidade. A parte inversa
# do teorema da o piso de BER que nenhum codigo daquela taxa consegue furar,
# por mais longo que seja o bloco (zero quando R <= C).
TABELA_ESQUEMAS["R > C"] = TABELA_ESQUEMAS["R"] > C
TABELA_ESQUEMAS["piso teorico da BER"] = [
    model.ber_floor_above_capacity(r, C) for r in TABELA_ESQUEMAS["R"]
]

salvar_tabela(TABELA_ESQUEMAS, "t4_passo7_comparacao_esquemas")
display(TABELA_ESQUEMAS)
print(f"capacidade C = {C:.4f} bits por uso do canal")
for esquema, linha in TABELA_ESQUEMAS.iterrows():
    if linha["R > C"]:
        print(f"{esquema}: R = {linha['R']:.4f} > C -> nenhum codigo dessa taxa
tem BER abaixo de "
              f"{linha['piso teorico da BER']:.4f} (medida: {linha['BER']:.4f})")

fig, ax = plt.subplots()
ax.bar(TABELA_ESQUEMAS.index, TABELA_ESQUEMAS["BER"], color=PALETA[:3])
ax.axhline(P_CANAL, ls="--", color="black", lw=1, label=f"p do canal = {P_CANAL}")
ax.set_ylabel("BER pos-decodificacao")
ax.set_title(f"Comparacao dos esquemas de codificacao (p = {P_CANAL})")
ax.legend()
salvar_fig("t4_passo7_comparacao_esquemas")
plt.show()
```

esquema	R	BER	R > C	piso teorico da BER
sem codificacao	1.000000	0.100093	True	0.10000
repeticao (n=3)	0.333333	0.028023	False	0.00000
Hamming(7,4)	0.571429	0.066832	True	0.00851

```
capacidade C = 0.5310 bits por uso do canal
sem codificacao: R = 1.0000 > C -> nenhum codigo dessa taxa tem BER abaixo de 0.1000
(medida: 0.1001)
Hamming(7,4): R = 0.5714 > C -> nenhum codigo dessa taxa tem BER abaixo de 0.0085 (m
edida: 0.0668)
```



Passo 8: Variar p

Repetir para varios p e observar como muda a vantagem de cada codigo. BER em escala log, com a curva exata do Hamming(7,4) e o ponto em que ele passa a piorar a transmissao.

In [9]:
```python
# Passo 8
P_VALORES = np.array([0.02, 0.05, 0.1, 0.15, 0.2, 0.3])

registros = []
for p in P_VALORES:
    recebido_direto_p = bsc(MSG, p, generator=GEN)
    ber_direto_p = np.mean(recebido_direto_p != MSG)

    recebido_rep_p = bsc(COD_REP, p, generator=GEN)
    decod_rep_p = model.repetition_decode(recebido_rep_p, N_REP)
    ber_rep_p = np.mean(decod_rep_p != MSG)

    recebido_ham_p = bsc(COD_HAM, p, generator=GEN)
    decod_ham_p = model.hamming74_decode(recebido_ham_p)
    ber_ham_p = np.mean(decod_ham_p != MSG)

    registros.append({"p": p, "esquema": "sem codificacao", "BER": ber_direto_p})
    registros.append({"p": p, "esquema": f"repeticao (n={N_REP})", "BER":
ber_rep_p})
```

```
    registros.append({"p": p, "esquema": "Hamming(7,4)", "BER": ber_ham_p})

DF_VARIANDO_P = pd.DataFrame(registros)

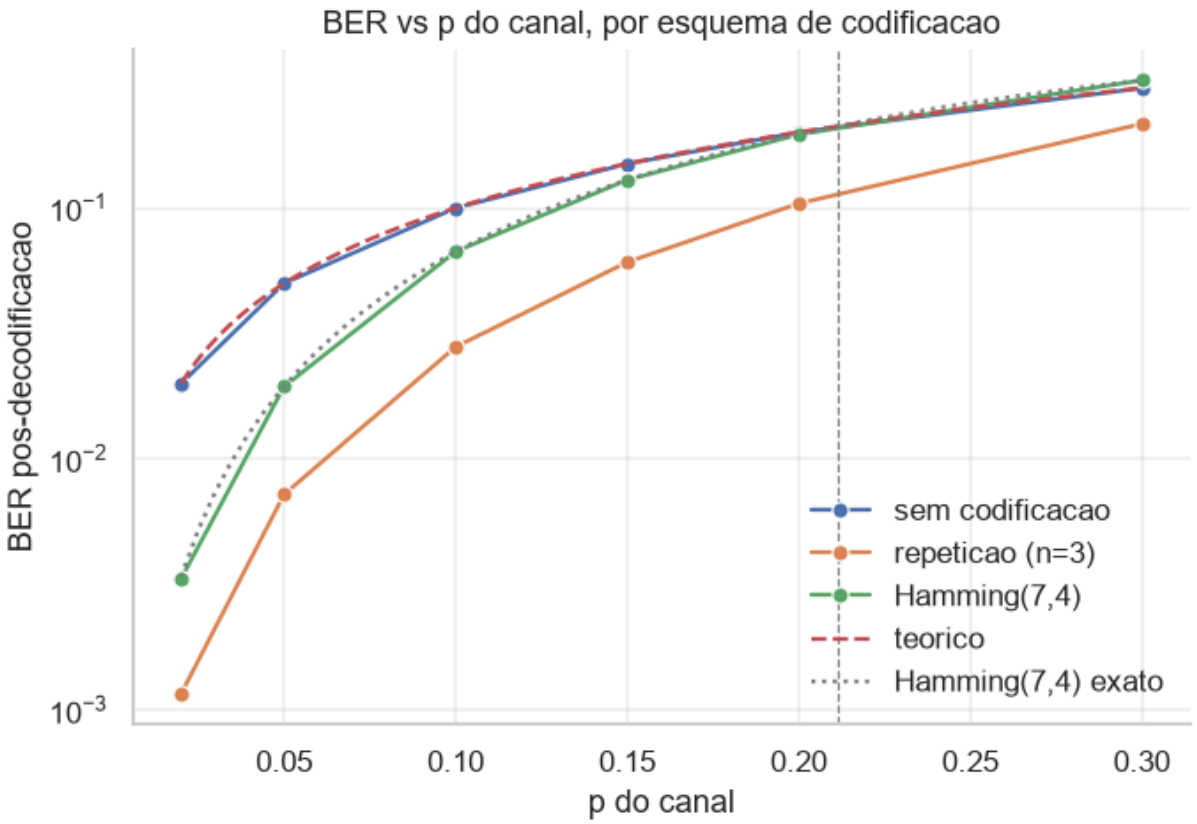
TABELA_VARIANDO_P = DF_VARIANDO_P.pivot(index="p", columns="esquema",
values="BER")
# conferencia: BER exata do Hamming(7,4), pelos 128 padroes de erro do bloco
TABELA_VARIANDO_P["Hamming(7,4) exato"] = [model.hamming74_ber_exata(p) for p in
TABELA_VARIANDO_P.index]
salvar_tabela(TABELA_VARIANDO_P, "t4_passo8_ber_vs_p")
display(TABELA_VARIANDO_P)

# A partir de que p o Hamming(7,4) passa a piorar a transmissao?
P_CRUZAMENTO = model.bissecao(lambda p: model.hamming74_ber_exata(p) - p, 0.15,
0.35)
print(f"Hamming(7,4) fica pior que sem codificacao a partir de p =
{P_CRUZAMENTO:.4f} "
      f"(7p = {7 * P_CRUZAMENTO:.2f} erros esperados por bloco)")

# BER em escala log, como sugere o Bloco 3: a diferenca entre os esquemas so
# aparece direito assim. A linha "teorico" e y = p (sem codificacao).
ax = plot_curvas(
    DF_VARIANDO_P, x="p", y="BER", hue="esquema", teorico=lambda p: p,
    xlabel="p do canal", ylabel="BER pos-decodificacao",
)
P_GRADE = np.linspace(P_VALORES.min(), P_VALORES.max(), 60)
ax.plot(P_GRADE, [model.hamming74_ber_exata(p) for p in P_GRADE], ls=":",
color="gray",
        label="Hamming(7,4) exato")
ax.axvline(P_CRUZAMENTO, ls="--", lw=0.8, color="gray")
ax.set_yscale("log")
ax.legend()
plt.title("BER vs p do canal, por esquema de codificacao")
salvar_fig("t4_passo8_ber_vs_p")
plt.show()
```

esquema	Hamming(7,4)	repeticao (n=3)	sem codificacao	Hamming(7,4) exato
p				
0.02	0.003342	0.001164	0.019980	0.003397
0.05	0.019431	0.007234	0.050074	0.019434
0.10	0.067010	0.027964	0.099874	0.066880
0.15	0.129248	0.060816	0.149775	0.129026
0.20	0.196014	0.104136	0.199982	0.196160
0.30	0.322186	0.216050	0.299838	0.321840

```
Hamming(7,4) fica pior que sem codificacao a partir de p = 0.2113 (7p = 1.48 erros e
sperados por bloco)
```



Passo 9: Promessa de Shannon

Aumentar n_rep = 3, 5, 7, 9, ...; ver a BER de bloco cair e relacionar com a distancia entre R e C. Conferir com a cauda binomial (`scipy.stats.binom`).

In [10]:
```
# Passo 9
N_REP_VALORES = np.array([3, 5, 7, 9, 11, 13, 15])

registros_n_rep = []
for n_rep in N_REP_VALORES:
    cod_n = model.repetition_encode(MSG, n_rep)
    recebido_n = bsc(cod_n, P_CANAL, generator=GEN)
    decod_n = model.repetition_decode(recebido_n, n_rep)

    r_n = 1 / n_rep
    ber_empirica_n = np.mean(decod_n != MSG)

    # decodificacao por maioria falha quando mais da metade dos n_rep bits do
    # bloco vira: cauda superior da binomial(n_rep, p) a partir da maioria.
    limiar = n_rep // 2
    ber_teorica_n = binom.sf(limiar, n_rep, P_CANAL)

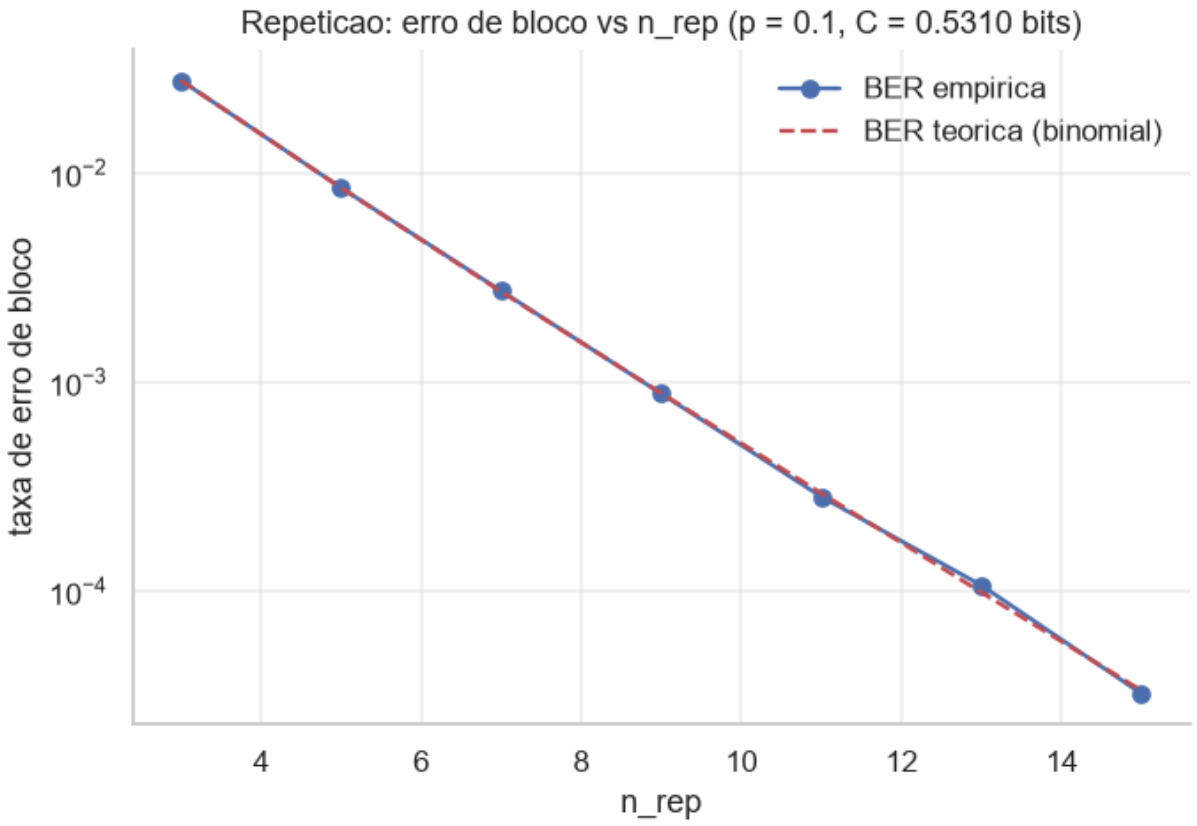
    registros_n_rep.append(
        {
            "n_rep": n_rep,
            "R": r_n,
            "C_menos_R": C - r_n,
            "BER_empirica": ber_empirica_n,
```

```
            "BER_teorica": ber_teorica_n,
        }
    )

DF_N_REP = pd.DataFrame(registros_n_rep).set_index("n_rep")
salvar_tabela(DF_N_REP, "t4_passo9_variando_n_rep")
display(DF_N_REP)

fig, ax = plt.subplots()
ax.plot(DF_N_REP.index, DF_N_REP["BER_empirica"], "o-", color=PALETA[0],
label="BER empirica")
ax.plot(DF_N_REP.index, DF_N_REP["BER_teorica"], "--", color=PALETA[3], label="BER
teorica (binomial)")
ax.set_yscale("log")
ax.set_xlabel("n_rep")
ax.set_ylabel("taxa de erro de bloco")
ax.set_title(f"Repeticao: erro de bloco vs n_rep (p = {P_CANAL}, C = {C:.4f}
bits)")
ax.legend()
salvar_fig("t4_passo9_variando_n_rep")
plt.show()
```

n_rep	R	C_menos_R	BER_empirica	BER_teorica
3	0.333333	0.197671	0.027955	0.028000
5	0.200000	0.331004	0.008569	0.008560
7	0.142857	0.388147	0.002748	0.002728
9	0.111111	0.419893	0.000886	0.000891
11	0.090909	0.440095	0.000285	0.000296
13	0.076923	0.454081	0.000107	0.000099
15	0.066667	0.464338	0.000032	0.000034



Passo 10: Cenario R > C

Escolher um canal bem mais ruidoso, com capacidade C bem menor que a taxa efetiva R de qualquer um dos tres esquemas; tentar reduzir a BER a um nivel aceitavel e documentar que nao e possivel. Reconstruir a imagem binarizada original lado a lado com a varredura de n_rep do Passo 9 (R < C, canal original) e os tres esquemas deste passo (R > C, canal ruidoso), para verificar visualmente o resultado final.

In [11]:
```
# Passo 10
P_CANAL_ALTO = 0.4  # canal bem mais ruidoso -> capacidade quase nula
C_ALTO = 1 - binary_entropy(P_CANAL_ALTO)
BER_ACEITAVEL = 0.01  # limiar arbitrario de "aceitavel" para a discussao

RECEBIDO_DIRETO_ALTO = bsc(MSG, P_CANAL_ALTO, generator=GEN)
ber_direto_alto = np.mean(RECEBIDO_DIRETO_ALTO != MSG)

RECEBIDO_REP_ALTO = bsc(COD_REP, P_CANAL_ALTO, generator=GEN)
DECOD_REP_ALTO = model.repetition_decode(RECEBIDO_REP_ALTO, N_REP)
ber_rep_alto = np.mean(DECOD_REP_ALTO != MSG)

RECEBIDO_HAM_ALTO = bsc(COD_HAM, P_CANAL_ALTO, generator=GEN)
DECOD_HAM_ALTO = model.hamming74_decode(RECEBIDO_HAM_ALTO)
ber_ham_alto = np.mean(DECOD_HAM_ALTO != MSG)

TABELA_R_MAIOR_C = pd.DataFrame(
    {
        "esquema": ["sem codificacao", f"repeticao (n={N_REP})", "Hamming(7,4)"],
```

```
        "R": [1.0, R_REP, R_HAM],
        "BER": [ber_direto_alto, ber_rep_alto, ber_ham_alto],
    }
).set_index("esquema")
TABELA_R_MAIOR_C["R > C"] = TABELA_R_MAIOR_C["R"] > C_ALTO
TABELA_R_MAIOR_C["BER aceitavel"] = TABELA_R_MAIOR_C["BER"] < BER_ACEITAVEL
TABELA_R_MAIOR_C["piso teorico da BER"] = [
    model.ber_floor_above_capacity(r, C_ALTO) for r in TABELA_R_MAIOR_C["R"]
]

salvar_tabela(TABELA_R_MAIOR_C, "t4_passo10_r_maior_c")
display(TABELA_R_MAIOR_C)

print(f"p do canal = {P_CANAL_ALTO}, capacidade C = {C_ALTO:.4f} bits")
print(f"nenhum dos tres esquemas chega perto de BER < {BER_ACEITAVEL} (todos com R
> C)")

# reconstrucao visual: só faz sentido se MSG veio da imagem real do Passo 3.
# MSG foi cortada para multiplo de 4 (Hamming); SOBRA_IMAGEM completa de volta
# o tamanho original antes do reshape.
if "IMAGEM_BINARIA" in globals():
    FORMATO = IMAGEM_BINARIA.shape
    PASSO = 4  # mostra 1 a cada 4 pixels por eixo, so para a figura renderizar
rapido

    def reconstroi(bits_decodificados):
        return np.concatenate([bits_decodificados, SOBRA_IMAGEM]).reshape(FORMATO)

    imagens = {"original": IMAGEM_BINARIA}

    # repete a varredura do Passo 9 (mesmo canal original, R < C) e reconstroi
    # a imagem recebida em cada n_rep -- mostra a melhora progressiva.
    for n_rep in N_REP_VALORES:
        n_rep = int(n_rep)
        cod_n = model.repetition_encode(MSG, n_rep)
        recebido_n = bsc(cod_n, P_CANAL, generator=GEN)
        decod_n = model.repetition_decode(recebido_n, n_rep)
        ber_n = np.mean(decod_n != MSG)
        imagens[f"P9: n={n_rep}\nBER={ber_n:.4f}"] = reconstroi(decod_n)

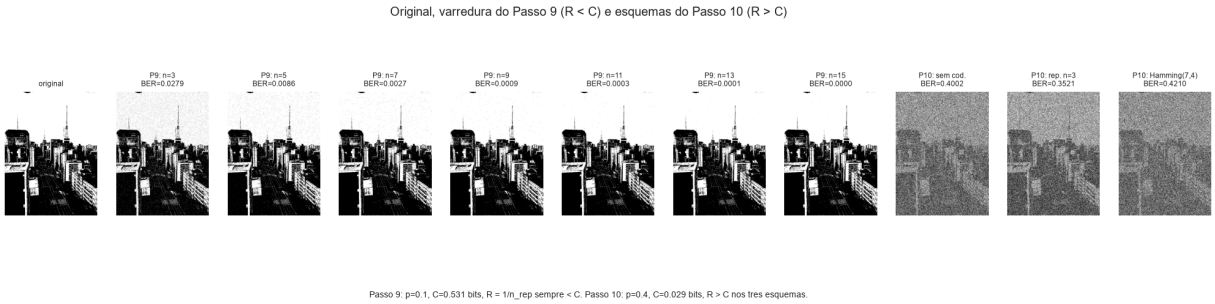
    # os tres esquemas do Passo 10, no canal ruidoso (R > C)
    imagens[f"P10: sem cod.\nBER={ber_direto_alto:.4f}"] =
reconstroi(RECEBIDO_DIRETO_ALTO)
    imagens[f"P10: rep. n={N_REP}\nBER={ber_rep_alto:.4f}"] =
reconstroi(DECOD_REP_ALTO)
    imagens[f"P10: Hamming(7,4)\nBER={ber_ham_alto:.4f}"] =
reconstroi(DECOD_HAM_ALTO)

    fig, eixos = plt.subplots(1, len(imagens), figsize=(2.1 * len(imagens), 4.5))
    for eixo, (titulo, imagem) in zip(eixos, imagens.items()):
        eixo.imshow(imagem[::PASSO, ::PASSO], cmap="gray")
        eixo.set_title(titulo, fontsize=8)
        eixo.axis("off")
    fig.suptitle("Original, varredura do Passo 9 (R < C) e esquemas do Passo 10 (R
> C)")
    fig.text(
```

```
        0.5,
        0.02,
        f"Passo 9: p={P_CANAL}, C={C:.3f} bits, R = 1/n_rep sempre < C. "
        f"Passo 10: p={P_CANAL_ALTO}, C={C_ALTO:.3f} bits, R > C nos tres
esquemas.",
        ha="center",
        fontsize=9,
    )
    salvar_fig("t4_passo10_imagens_resultantes")
    plt.show()
else:
    print("sem imagem real disponivel (mensagem sintetica) -- comparacao visual
pulada.")
```

esquema	R	BER	R > C	BER aceitavel	piso teorico da BER
sem codificacao	1.000000	0.400243	True	False	0.400000
repeticao (n=3)	0.333333	0.352111	True	False	0.327982
Hamming(7,4)	0.571429	0.420998	True	False	0.368051

```
p do canal = 0.4, capacidade C = 0.0290 bits
nenhum dos tres esquemas chega perto de BER < 0.01 (todos com R > C)
```



Passo 10 (continuacao): tentar de fato reduzir a BER no canal ruidoso

O roteiro pede para *tentar* reduzir a taxa de erro a um nivel aceitavel com os codigos disponiveis. Com a repeticao isso e possivel, mas so pagando taxa: aqui n_rep cresce no canal com p = 0.40 ate a BER ficar aceitavel, e cada ponto e comparado com o piso teorico de qualquer codigo da mesma taxa.

In [12]:
```
# Passo 10 (continuacao)
# Simula num trecho da mensagem (N_SUB bits) para nao gerar bilhoes de usos do
# canal; com BER ~ 0.01, 100 mil bits ainda dao ~1000 erros por ponto, e a
# cauda binomial confere cada um.
N_SUB = 100_000
MSG_SUB = MSG[:N_SUB]
N_REP_ALTO = [3, 9, 19, 33, 35, 51, 75, 101, 133, 151]

registros_alto = []
```

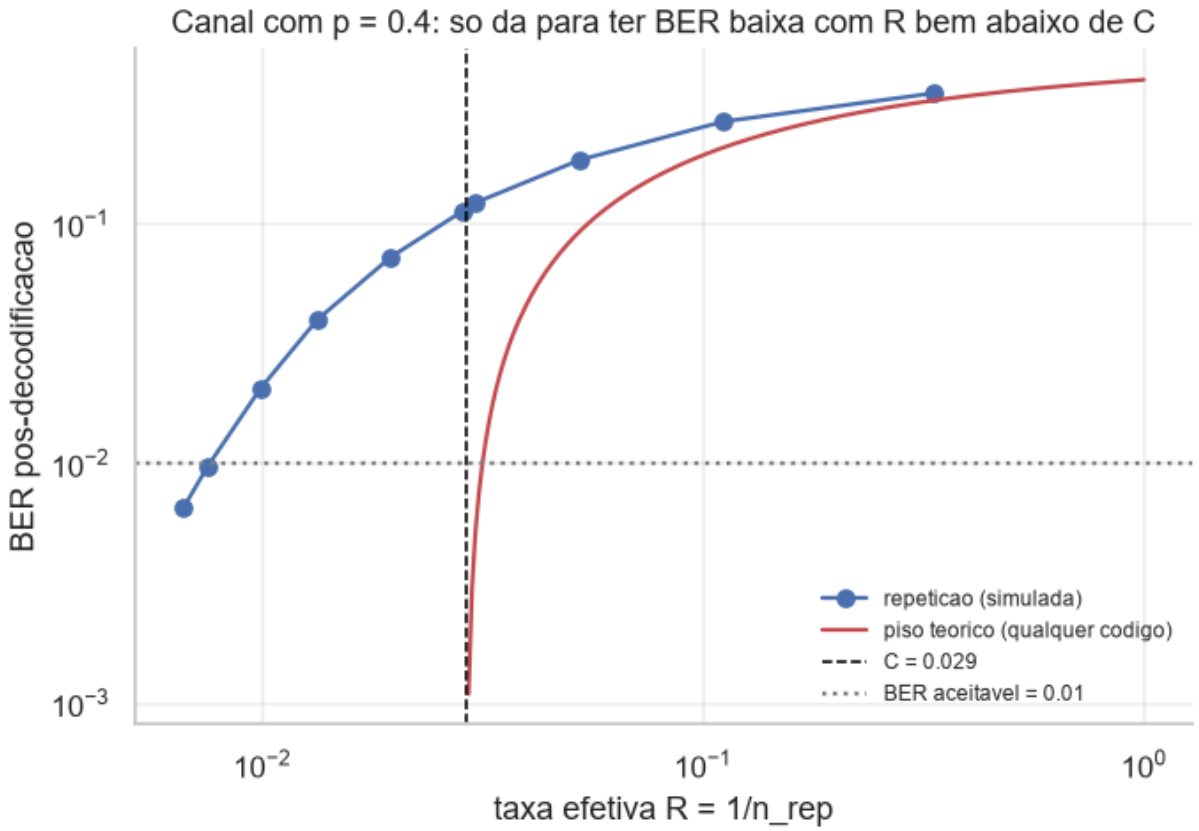
```
for n_rep in N_REP_ALTO:
    recebido = bsc(model.repetition_encode(MSG_SUB, n_rep), P_CANAL_ALTO,
generator=GEN)
    r = 1 / n_rep
    registros_alto.append({
        "n_rep": n_rep,
        "R": r,
        "R > C": r > C_ALTO,
        "BER empirica": np.mean(model.repetition_decode(recebido, n_rep) !=
MSG_SUB),
        "BER teorica (binomial)": binom.sf(n_rep // 2, n_rep, P_CANAL_ALTO),
        "piso teorico": model.ber_floor_above_capacity(r, C_ALTO),
    })
DF_REP_ALTO = pd.DataFrame(registros_alto).set_index("n_rep")
salvar_tabela(DF_REP_ALTO, "t4_passo10_varredura_canal_ruidoso")
display(DF_REP_ALTO)

N_REP_MAX_ACIMA = max(n for n in range(1, int(np.ceil(1 / C_ALTO)) + 1, 2) if 1 /
n > C_ALTO)  # maior impar com R > C
N_REP_ACEITAVEL = next(n for n in range(3, 1001, 2) if binom.sf(n // 2, n,
P_CANAL_ALTO) < BER_ACEITAVEL)
print(f"enquanto R > C (n_rep <= {N_REP_MAX_ACIMA}), a menor BER da repeticao e "
      f"{binom.sf(N_REP_MAX_ACIMA // 2, N_REP_MAX_ACIMA, P_CANAL_ALTO):.4f}")
print(f"menor n_rep com BER < {BER_ACEITAVEL}: {N_REP_ACEITAVEL} -> R = {1 /
N_REP_ACEITAVEL:.4f} "
      f"= {1 / N_REP_ACEITAVEL / C_ALTO:.2f} x C")

fig, ax = plt.subplots()
ax.plot(DF_REP_ALTO["R"], DF_REP_ALTO["BER empirica"], "o-", color=PALETA[0],
label="repeticao (simulada)")
R_GRADE = np.geomspace(1 / 200, 1, 300)
piso = np.array([model.ber_floor_above_capacity(r, C_ALTO) for r in R_GRADE])
ax.plot(R_GRADE, np.where(piso > 0, piso, np.nan), color=PALETA[3], label="piso
teorico (qualquer codigo)")
ax.axvline(C_ALTO, ls="--", color="black", lw=1, label=f"C = {C_ALTO:.3f}")
ax.axhline(BER_ACEITAVEL, ls=":", color="gray", label=f"BER aceitavel =
{BER_ACEITAVEL}")
ax.set_xscale("log")
ax.set_yscale("log")
ax.set_xlabel("taxa efetiva R = 1/n_rep")
ax.set_ylabel("BER pos-decodificacao")
ax.set_title(f"Canal com p = {P_CANAL_ALTO}: so da para ter BER baixa com R bem
abaixo de C")
ax.legend(fontsize=8)
salvar_fig("t4_passo10_varredura_canal_ruidoso")
plt.show()
```

n_rep	R	R > C	BER empirica	BER teorica (binomial)	piso teorico
3	0.333333	True	0.35151	0.352000	0.327982
9	0.111111	True	0.26771	0.266568	0.208474
19	0.052632	True	0.18542	0.186092	0.093505
33	0.030303	True	0.12226	0.121099	0.004476
35	0.028571	False	0.11367	0.114313	0.000000
51	0.019608	False	0.07297	0.073529	0.000000
75	0.013333	False	0.03991	0.039577	0.000000
101	0.009901	False	0.02055	0.020897	0.000000
133	0.007519	False	0.00973	0.009785	0.000000
151	0.006623	False	0.00653	0.006443	0.000000

```
enquanto R > C (n_rep <= 33), a menor BER da repeticao e 0.1211
menor n_rep com BER < 0.01: 133 -> R = 0.0075 = 0.26 x C
```



Relatorio

Consolidar resultados, tabelas e figuras em `relatorio.md` (figuras salvas em `figuras/`).